

graphical user interfaces (GUI) using Tcl. They can be extended and embedded in other applications easily.

Eclipse: The Eclipse platform can be used to develop applications in Java and other programming languages (by means of various plug-ins) including Ada, C, C++, COBOL, FORTRAN, Haskell, JavaScript, Perl, PHP, Python, R, Ruby (including the Ruby on Rails framework), etc. The Eclipse SDK includes the Eclipse Java Development Tools (JDT), a built-in incremental Java compiler and Java source files.

NetBeans: NetBeans is an integrated development environment (IDE) for developing applications primarily with Java and with other languages like PHP, C/C++, and HTML5. It can run on Windows, OS X, Linux, Solaris and other platforms supporting a compatible JVM. The NetBeans IDE includes the NetBeans profiler, GUI designer tool, and the NetBeans Java Script editor. The NetBeans IDE bundles are tailored to specific development needs like standalone, Web and mobile applications. The NetBeans Platform allows applications to be developed from a set of modular software components called modules, which can be extended by third party developers.

Qt: Qt is a cross-platform application framework that is widely used for developing application software with a graphical user interface (GUI), and for non-GUI programs such as command-line tools and consoles for servers. Qt uses standard C++. Qt can also be used in several other programming languages via language bindings. It works on a variety of platforms like Windows, OS X, X11, Embedded Linux, Wayland, QNX / BlackBerry 10, Android, iOS and VxWorks.

For the sake of simplicity, let us narrow down our search to a list of Java IDEs that come with handy plug-ins for commercial and personal use. Let us target our mission at the Eclipse IDE, which is considered by many to be the best Java development tool.

Eclipse architecture

All of us are given the freedom to select the best tool for any task. However, most software models only integrate with selective programs owing to dependencies and technology limitations. Additionally, human intervention is needed to manage file and data transfer between tools.

Eclipse is a platform that has been designed to build integrated Web and application development tools. In this arrangement, loose coupling is achieved through the mechanism of extensions and extension points.

The Eclipse platform uses the model of a common workbench. The tools that we develop can plug into the workbench using well defined hooks called extension points.

A workbench is the main Eclipse window, which opens when you start the Eclipse IDE. Inside the workbench is a workbench window, the frame that holds the menu bar, a toolbar, a status line, short-cut bar and pages. The page referred to here is the workbench page, which supplies the window contents, namely, the views and editors.

In a real world situation, you navigate to various screens and expect a customised look and feel for each screen. This

functionality is achieved through ‘perspectives’. A perspective acts as a container for views and editors. It defines the initial set of views to be shown along with their layouts. Additionally, perspectives control menus and tool bars that appear on the screen. Although perspectives appear to contain views and editors, they only own the layout. It is the workbench page that owns the views and editors, and maintains a reference of how many perspectives use each view.

If you compare a workbench to a book, then the perspectives are pages of the book. Like each page, a perspective possesses a unique combination of details.

Now that the basic organisation of a workbench is clear, how do I hook my tool?

Before we start, you need to understand the term ‘plug-in’. It is a structured bundle of code that contributes a function to the system. The Eclipse platform in itself is a built-in layer of plug-ins, each one defining extensions to extension points of lower-level plug-ins. When a plug-in wants to add other plug-ins to extend portions of its functionality, it will declare an extension point. The extension point declares a contract that extensions must conform to. This extension model allows the developer to expand the functionality with the basic tooling platform.

Eclipse uses OSGi as the basis for its plug-in system. The OSGi (Open Services Gateway initiative) technology is a set of specifications that define a dynamic component system for Java. Components in the form of bundles can be remotely installed, started, stopped, updated and uninstalled without requiring a reboot. The OSGi specifications enable components to hide their implementations from other components while communicating through services, which are objects that are specifically shared between components. The OSGi has a layered model that is depicted in Figure 1.

A module (also called a bundle) here is a ready-to-deploy application (executable file/ jar file), the jar file of our plug-in. These extensions are bound to a lifecycle. The OSGi framework offers APIs to manage the lifecycle of modules like *install*, *start*, *stop*, *refresh*, *update*, and *uninstall*. A bundle registers zero or more services for interoperability. The service layer connects the bundles dynamically by the *Publish-Find-Bind* model.

Thus the platform runtime core implements the runtime engine that starts the platform base and dynamically discovers and runs plug-ins. The platform maintains a registry of installed plug-ins and the functionality they provide.

Eclipse Juno has introduced new features and has brought in a high level of flexibility to the programming. Equinox is one implementation of OSGi and is used by the Eclipse platform. The Equinox runtime provides the necessary framework to run a modular Eclipse application.

What is RCP?

Eclipse RCP is a platform for building and deploying rich client applications. It is based on the familiar Eclipse plug-in architecture. RCP (Rich Client Platform) opens the door to achieving desktop applications written in the